Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

DYNAMIC TRAFFIC SIGNAL OPTIMIZATION SIMULATOR

Team ID: DSCPP-III-2026-T032	
Team member 1 (Team Lead)	*Singh, Yashveer – 2510011338* *2510011338@geu.ac.in* 
Team member 2	*Bhatnagar, Samanyu – 2510360131* *2510360131@geu.ac.in* 
Team member 3	*Alam, Faiz – 2510011503* *2510011503@geu.ac.in* 
Team member 4	*Kadiyan, Vansh – 2510380251* *2510380251@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

The main goal of this project is to develop a "Dynamic Traffic Signal Optimization Simulator," a robust software system that models real-world intersection complexities. The simulation aims to visualize and dynamically optimize traffic flow across a grid of interconnected intersections. A primary objective is the implementation of an intelligent traffic light control system that actively polls queue densities and automatically overrides standard light cycles to prioritize emergency vehicles.

Additionally, the project seeks to simulate unpredictable real-world scenarios—such as vehicular collisions, reckless driving, and lane blockages—to test the resilience of dynamic pathfinding algorithms. When an accident occurs and blocks a specific path, the simulator recalculates vehicle routes in real-time using graph traversal techniques, allowing AI-driven cars to perform maneuvers like U-turns to seek alternative routes. Ultimately, this project serves as a comprehensive visual and algorithmic model for evaluating smart city traffic management strategies and autonomous vehicle navigation in the presence of dynamic obstacles.

Updated Project Approach and Architecture

The base simulation engine is the `IntersectionSimulator` class that controls a 2x3 grid of `Intersection` objects, each containing four `TrafficLane` instances.

Libraries Used: The visual layer, input handling and the geometric math is provided by the **Raylib** library.

C++ Standard Template Library (vectors, maps, queues): Dynamic entities and state tracking.

DS & Algo: Counter for Vehicles in Lanes (Dynamic payload FIFO Queue): `queue.h/queue.c`. Processing cars in FIFO queues—in order—only as they come to a halt at signals.

The loop exemplifies a frame-based architecture (60 FPS, but your mileage may vary) with distinct boundaries between state calculating `updateLogic` and drawing `drawLogic`.

Visually, it uses the isometric pseudo-3D parallax rendering system that we previously created, stacking generated buildings atop one another as 2D rectangles using a scale based on a focal point from the camera.

Core Algorithms:

Pathfinding & Rerouting: All the junctions that intersect are directed graphs. BFS is used to get best edges. In case of block on a lane by a `CrashEvent`, vehicles query the graph and navigate through realtime dynamic routing.

Kinematics: We computationally calculate and filter-in a mathematically basis of vehicle trajectory (turns/dynamic U-turns)—employing continuous Bézier curves.

Collision Avoidance: And also specifics on Collision avoidance: predictive braking — collision logic, axis-aligned bounding box (AABB) & circular for objects in front use speed multipliers then when playing overlap another reckless vehicle will crash.

Signal Logic: Intersections use Round robin cycle + priority by green-light. When it have a Queued Data structure with high `emergencyCount`.

Tasks Completed

Task Completed	Team Member
• Implementation Queue OopCore – C-based CSCS instruct execo domain; kernel, page.h, queue.c) for vehicle tracking.	• Yashveer Singh
• Raylib 2D rendering engine.	• Samanyu Bhatnagar
• Parallax Background in pseudo 3D.	• Faiz Alam
• Integrated BFS pathfinding algorithms dynamic rerouting on accidents.	• Vansh Kadiyan

Challenges/Roadblocks

One major challenge we faced was smoothing vehicle turning paths at intersections. We successfully resolved this by mathematically implementing continuous cubic Bézier curves to calculate fluid transition frames instead of sharp angle turns.

Another significant roadblock has been managing dynamic memory for the custom queue structures alongside Raylib's continuous 60 FPS frame loop. To solve this, we are actively structuring careful garbage collection of `Vehicle` and `CrashEvent` instances to guarantee the prevention of memory leaks during prolonged simulations.

Lastly, we are currently facing difficulties in tuning the collision detection to accurately trigger predictive braking without causing infinite traffic jams. We plan to solve this by introducing a weighted distance heuristic and fine-tuning the parameter adjustments over extensive test runs.

Tasks Pending

Task Pending	Team Member (to complete the task)
• Optimize the BFS graph traversal logic for potential larger grid networks.	• Yashveer Singh & Faiz Alam
• Add a graphical overlay/dashboard for real-time traffic statistics.	• Samanyu Bhatnagar & Vansh Kadiyan

Project Outcome/Deliverables

The key deliverables of this project include a fully functional, standalone executable traffic simulation application developed in C/C++ using the Raylib framework. The primary outcome is a robust modeling tool capable of simulating a 2x3 grid of dynamic intersections.

Key deliverables feature: 1) A custom C-based dynamic FIFO queue system for vehicle management. 2) An intelligent traffic light control system that overrides standard cycles for emergency vehicles. 3) A real-time routing engine utilizing Breadth-First Search (BFS) to dynamically divert traffic around simulated accidents. 4) A polished visual interface utilizing pseudo-3D parallax rendering to provide an immersive top-down view of the traffic grid. Overall, the outcome is a comprehensive platform for testing and visualizing smart city traffic management strategies.

Progress Overview

Overall, approximately 85% of the project is completed, placing us ahead of our general development schedule. The core mechanics—including the custom queue data structures, Raylib 2D/pseudo-3D rendering pipeline, and the BFS-based dynamic pathfinding algorithms—are fully implemented and functioning smoothly at 60 FPS. We are currently ahead of schedule regarding the visual polish and Bézier curve vehicle kinematics, which were completed earlier than anticipated.

However, we are slightly behind schedule on fine-tuning the collision detection system; predictive braking occasionally triggers too early, leading to unintended traffic bottlenecks. Our immediate focus is resolving this collision tuning and finalizing the graphical dashboard for real-time traffic statistics before the final submission deadline.

Codebase Information

Repository Link: `https://github.com/syashveer940-cyber/TrafficSignalSimulator`

Active Branch: `main`

Important Commits:

- `[e4a2b1c]` Initialized C Queue structures (`queue.h`, `queue.c`) and memory management.
- `[7f8d9e0]` Integrated Raylib rendering loop, 60 FPS frame limiter, and parallax backgrounds.
- `[1a2b3c4]` Added BFS graph traversal for dynamic collision rerouting.
- `[9c8b7a6]` Implemented emergency vehicle signal override logic.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
1. Queue Memory Allocation	Pass	Monitored for memory leaks over 60+ minute continuous runs.
2. Emergency Signal Override	Pass	Validated that queues with `emergencyCount > 0` successfully force green lights.
3. Collision & Rerouting	Pass	Confirmed BFS effectively recalculates paths when a `CrashEvent` blocks a lane.

Deliverables Progress

The current status of our key project deliverables is as follows:

- **C-based dynamic FIFO queue system:** Completed.
- **Intelligent traffic light control (emergency overrides):** Completed.
- **Real-time BFS routing engine (accident avoidance):** Completed.
- **Visual interface (pseudo-3D parallax, Raylib):** Completed.
- **Collision detection & predictive braking tuning:** In Progress.
- **Graphical dashboard for real-time traffic statistics:** Pending.